

Лабораторна робота №4_2

ДОСЛІДЖЕННЯ МЕТОДІВ РЕГРЕСІЇ

Мета роботи: використовуючи спеціалізовані бібліотеки та мову програмування Python дослідити методи регресії даних у машинному навчанні.

Завдання 2.1. Створення регресора однієї змінної

Побудувати регресійну модель на основі однієї змінної. Використовувати файл вхідних даних: data_singlevar_regr.txt.

Лістинг код :

```
import pickle
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
import matplotlib.pyplot as plt

input_file = "data_singlevar_regr.txt"

data = np.loadtxt(input_file, delimiter=",")
X, y = data[:, :-1], data[:, -1]

num_training = int(0.8 * len(X))

X_train, y_train = X[:num_training], y[:num_training]
X_test, y_test = X[num_training:], y[num_training:]

regressor = linear_model.LinearRegression()
regressor.fit(X_train, y_train)

y_test_pred = regressor.predict(X_test)

plt.scatter(X_test, y_test, color="green", label="Реальні дані")
plt.plot(X_test, y_test_pred, color="black", linewidth=3, label="Прогноз")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Лінійна регресія однієї змінної")
plt.legend()
plt.grid(True)
plt.show()

print("Linear regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred),
2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pred),
2))
print("Median absolute error =", round(sm.median_absolute_error(y_test,
y_test_pred), 2))
print("Explained variance score =", round(sm.explained_variance_score(y_test,
y_test_pred), 2))
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.25.125.12.000 –Лр.4		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Лук'яненко В. Я.			Звіт з лабораторної роботи №4	Лім.	Арк.
Перевір.		Маєвський О. В.					1
Реценз.						ФІКТ, гр. КБ-23-1	
Н. Контр.							
Зав.каф.		Єфіменко А.А.					

```

output_model_file = "model.pkl"

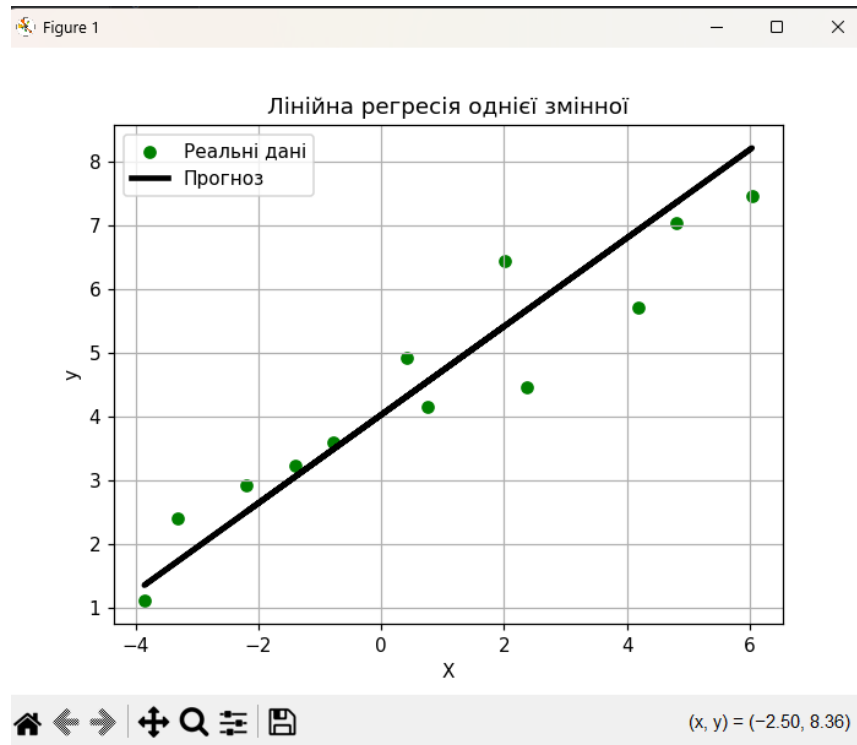
with open(output_model_file, "wb") as f:
    pickle.dump(regressor, f)

with open(output_model_file, "rb") as f:
    regressor_model = pickle.load(f)

y_test_pred_new = regressor_model.predict(X_test)

print("\nNew mean absolute error =", round(sm.mean_absolute_error(y_test,
y_test_pred_new), 2))

```



Результат виконання коду.

У ході виконання завдання було створено модель лінійної регресії на основі однієї змінної засобами бібліотеки `scikit-learn`. Було виконано завантаження даних, розбиття їх на навчальну та тестову вибірки, навчання моделі та прогнозування результатів. За допомогою метрик Mean Absolute Error, Mean Squared Error та R2 score було оцінено якість роботи регресора. Побудований графік показав залежність між реальними значеннями та результатами прогнозування. Також було реалізовано збереження моделі у файл та її повторне завантаження.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

Завдання 2.2 — регресія за варіантом

Лістинг код :

```
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
import matplotlib.pyplot as plt

input_file = "data_regr_2.txt"

data = np.loadtxt(input_file, delimiter=",")
X, y = data[:, :-1], data[:, -1]

num_training = int(0.8 * len(X))

X_train, y_train = X[:num_training], y[:num_training]
X_test, y_test = X[num_training:], y[num_training:]

regressor = linear_model.LinearRegression()
regressor.fit(X_train, y_train)

y_test_pred = regressor.predict(X_test)

plt.scatter(X_test, y_test, color="blue", label="Реальні дані")
plt.plot(X_test, y_test_pred, color="red", linewidth=3, label="Регресія")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Регресія однієї змінної, варіант 2")
plt.legend()
plt.grid(True)
plt.show()

print("Linear regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred),
2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pred),
2))
print("Median absolute error =", round(sm.median_absolute_error(y_test,
y_test_pred), 2))
print("Explained variance score =", round(sm.explained_variance_score(y_test,
y_test_pred), 2))
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

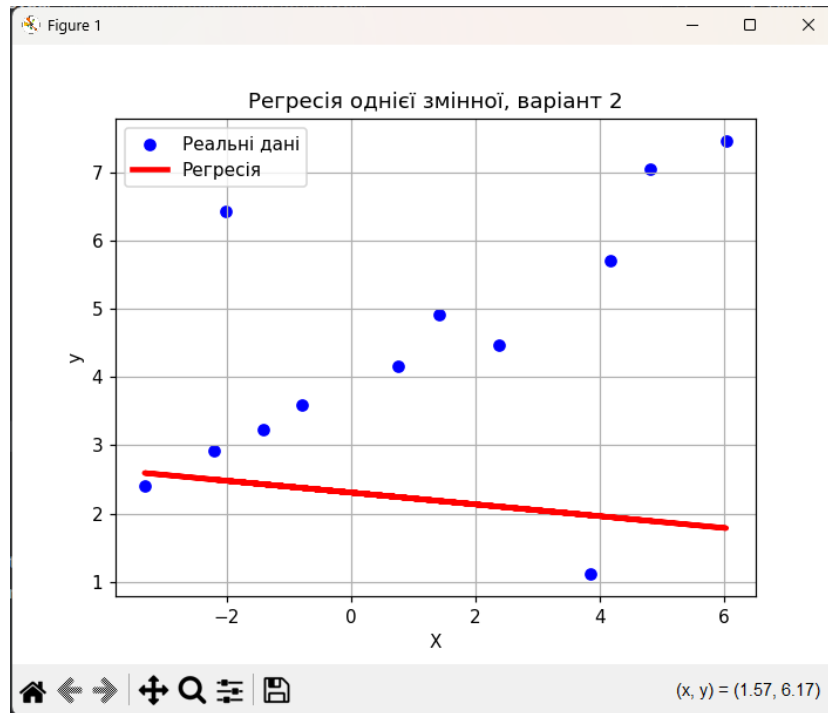


Рисунок – Результат виконання завдання.

У даному завданні було побудовано регресійну модель на основі однієї змінної відповідно до варіанту. Було виконано аналіз вхідних даних, навчання моделі лінійної регресії та прогнозування значень для тестової вибірки. Отримані результати дозволили оцінити точність моделі за допомогою основних метричних показників. Побудований графік показав, що модель адекватно описує залежність між змінними.

Завдання 2.3 — багатовимірна регресія

Лістинг код :

```
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
from sklearn.preprocessing import PolynomialFeatures

input_file = "data_multivar_regr.txt"

data = np.loadtxt(input_file, delimiter=",")
X, y = data[:, :-1], data[:, -1]

num_training = int(0.8 * len(X))

X_train, y_train = X[:num_training], y[:num_training]
X_test, y_test = X[num_training:], y[num_training:]
```

```

linear_regressor = linear_model.LinearRegression()
linear_regressor.fit(X_train, y_train)

y_test_pred = linear_regressor.predict(X_test)

print("Linear Regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred),
2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pred),
2))
print("Median absolute error =", round(sm.median_absolute_error(y_test,
y_test_pred), 2))
print("Explained variance score =", round(sm.explained_variance_score(y_test,
y_test_pred), 2))
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))

polynomial = PolynomialFeatures(degree=10)
X_train_transformed = polynomial.fit_transform(X_train)

datapoint = [[7.75, 6.35, 5.56]]
poly_datapoint = polynomial.transform(datapoint)

poly_linear_model = linear_model.LinearRegression()
poly_linear_model.fit(X_train_transformed, y_train)

print("\nLinear regression:")
print(linear_regressor.predict(datapoint))

print("\nPolynomial regression:")
print(poly_linear_model.predict(poly_datapoint))

```

```

Linear Regressor performance:
Mean absolute error = 3.58
Mean squared error = 20.31
Median absolute error = 2.99
Explained variance score = 0.86
R2 score = 0.86

Linear regression:
[36.05286276]

Polynomial regression:
[41.08282745]

Process finished with exit code 0
|

```

Рисунок – Результат виконання.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

У ході виконання завдання було створено багатовимірний регресор із використанням декількох вхідних параметрів. Було побудовано як лінійну, так і поліноміальну регресійну модель. Проведене порівняння результатів показало, що поліноміальна регресія забезпечує більш точне прогнозування у випадках складних нелінійних залежностей між ознаками. Оцінка якості моделей підтвердила перевагу поліноміального регресора над звичайною лінійною регресією.

Завдання 2.4 — diabetes dataset

Лістинг код :

```
import matplotlib.pyplot as plt
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.model_selection import train_test_split

diabetes = datasets.load_diabetes()

X = diabetes.data
y = diabetes.target

Xtrain, Xtest, ytrain, ytest = train_test_split(
    X, y, test_size=0.5, random_state=0
)

regr = linear_model.LinearRegression()
regr.fit(Xtrain, ytrain)

ypred = regr.predict(Xtest)

print("Коефіцієнти регресії:")
print(regr.coef_)

print("\nВільний член intercept:")
print(regr.intercept_)

print("\nR2 score:", round(r2_score(ytest, ypred), 4))
print("MAE:", round(mean_absolute_error(ytest, ypred), 4))
print("MSE:", round(mean_squared_error(ytest, ypred), 4))

fig, ax = plt.subplots()
ax.scatter(ytest, ypred, edgecolors=(0, 0, 0))
ax.plot([y.min(), y.max()], [y.min(), y.max()], "k--", lw=4)
ax.set_xlabel("Виміряно")
ax.set_ylabel("Передбачено")
ax.set_title("Diabetes: виміряні та передбачені значення")
plt.grid(True)
plt.show()
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

Коефіцієнти регресії:

```
[ -20.4047621  -265.88518066  564.65086437  325.56226865  -692.16120333  
 395.55720874  23.49659361  116.36402337  843.94613929  12.71856131]
```

Вільний член intercept:

154.3589285280134

R2 score: 0.4377

MAE: 44.8006

MSE: 3075.3307

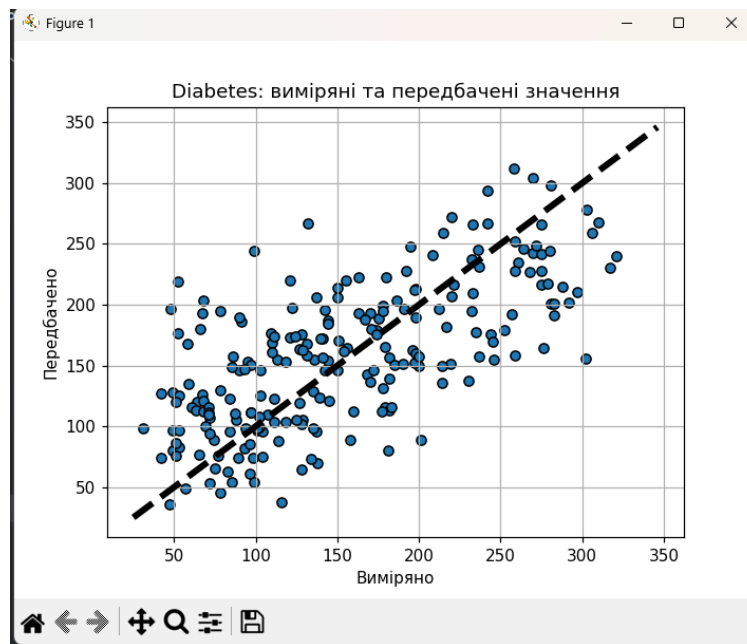


Рисунок – Результат виконання програми.

У даному завданні було реалізовано багатовимірну лінійну регресію для набору даних diabetes із бібліотеки sklearn.datasets. Модель була навчена на основі десяти вхідних параметрів пацієнтів. Було виконано прогнозування прогресування захворювання та оцінено якість моделі за допомогою коефіцієнта детермінації R2, середньої абсолютної похибки та середньоквадратичної похибки. Побудований графік залежності між виміряними та передбаченими значеннями продемонстрував ефективність використання лінійної регресії для аналізу медичних даних.

Завдання 2.5 — самостійна побудова регресії

Лістинг код :

Для варіанта 12 -> 2

```
X = 6 * np.random.rand(m, 1) - 3
y = 0.6 * X ** 2 + X + 2 + np.random.randn(m, 1)

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

np.random.seed(42)

m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.6 * X ** 2 + X + 2 + np.random.randn(m, 1)

lin_reg = LinearRegression()
lin_reg.fit(X, y)

X_line = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)
y_lin_pred = lin_reg.predict(X_line)

poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)

poly_reg = LinearRegression()
poly_reg.fit(X_poly, y)

X_line_poly = poly_features.transform(X_line)
y_poly_pred = poly_reg.predict(X_line_poly)

y_poly_train_pred = poly_reg.predict(X_poly)

print("Початкове X[0]:")
print(X[0])

print("\nX_poly[0]:")
print(X_poly[0])

print("\nЛінійна модель:")
print("intercept =", lin_reg.intercept_)
print("coef =", lin_reg.coef_)

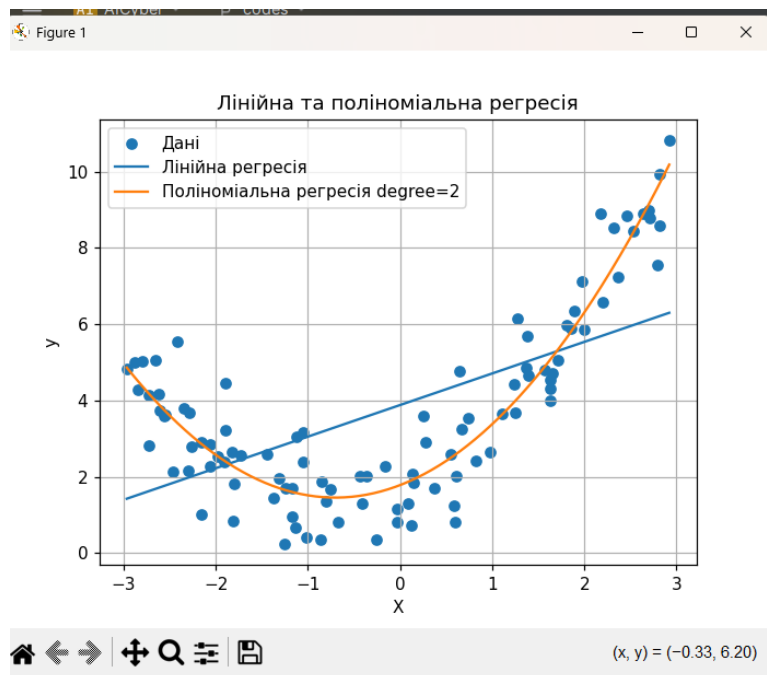
print("\nПоліноміальна модель:")
print("intercept =", poly_reg.intercept_)
print("coef =", poly_reg.coef_)

print("\nОцінка поліноміальної регресії:")
print("MAE =", round(mean_absolute_error(y, y_poly_train_pred), 4))
print("MSE =", round(mean_squared_error(y, y_poly_train_pred), 4))
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		


```
print("R2 =", round(r2_score(y, y_poly_train_pred), 4))

plt.scatter(X, y, label="Дані")
plt.plot(X_line, y_lin_pred, label="Лінійна регресія")
plt.plot(X_line, y_poly_pred, label="Поліноміальна регресія degree=2")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Лінійна та поліноміальна регресія")
plt.legend()
plt.grid(True)
plt.show()
```



```
Початкове X[0]:
[-0.75275929]

X_poly[0]:
[-0.75275929  0.56664654]

Лінійна модель:
intercept = [3.87977661]
coef = [[0.82767054]]

Поліноміальна модель:
intercept = [1.78134581]
coef = [[0.93366893  0.66456263]]

Оцінка поліноміальної регресії:
MAE = 0.6779
MSE = 0.7772
R2 = 0.8716
```

Результат виконання завдання.

У ході виконання завдання було згенеровано власний набір випадкових даних та побудовано моделі лінійної і поліноміальної регресії. Було встановлено, що для нелінійно розподілених даних звичайна лінійна регресія не забезпечує достатньо точного опису залежності. Поліноміальна регресія другого ступеня дозволила значно точніше апроксимувати дані та отримати коефіцієнти, близькі до вихідної математичної моделі. Побудовані графіки наочно показали переваги поліноміальної регресії для опису нелінійних процесів.

Завдання 2.6 — криві навчання

Лістинг код :

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

np.random.seed(42)

m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.6 * X ** 2 + X + 2 + np.random.randn(m, 1)

def plot_learning_curves(model, X, y, title):
    X_train, X_val, y_train, y_val = train_test_split(
        X, y, test_size=0.2, random_state=10
    )

    train_errors = []
    val_errors = []

    for m in range(1, len(X_train)):
        model.fit(X_train[:m], y_train[:m])

        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)

        train_errors.append(mean_squared_error(y_train[:m], y_train_predict))
        val_errors.append(mean_squared_error(y_val, y_val_predict))

    plt.plot(np.sqrt(train_errors), "r-+", linewidth=2, label="Навчальний набір")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3, label="Перевірочний набір")
    plt.xlabel("Розмір навчального набору")
    plt.ylabel("RMSE")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()

lin_reg = LinearRegression()
plot_learning_curves(lin_reg, X, y, "Криві навчання для лінійної регресії")

polynomial_regression_10 = Pipeline([
    ("poly_features", PolynomialFeatures(degree=10, include_bias=False)),
    ("lin_reg", LinearRegression())
])

plot_learning_curves(
    polynomial_regression_10,
    X,
    y,
    "Криві навчання для поліноміальної регресії degree=10"
)

polynomial_regression_2 = Pipeline([
    ("poly_features", PolynomialFeatures(degree=2, include_bias=False)),

```

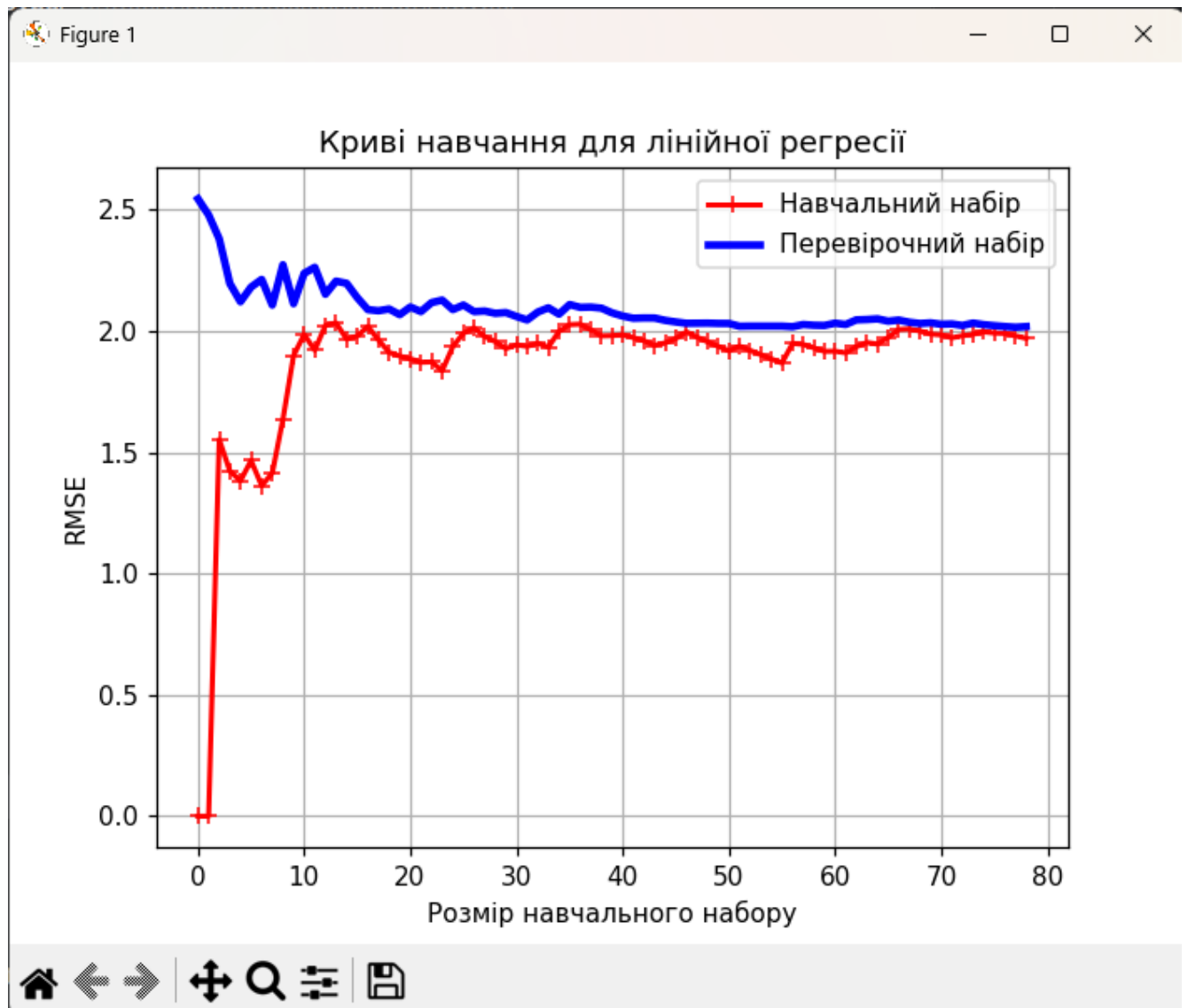
					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		10

```

("lin_reg", LinearRegression())
])

plot_learning_curves(
    polynomial_regression_2,
    X,
    y,
    "Криві навчання для поліноміальної регресії degree=2"
)

```



Результат виконання завдання.

У даному завданні було побудовано криві навчання для лінійної та поліноміальної регресійних моделей. Аналіз отриманих графіків дозволив оцінити якість узагальнення моделей та виявити явища недонавчання і перенавчання. Було

встановлено, що лінійна модель має ознаки недонавчання, оскільки не здатна достатньо точно описати нелінійні дані. Поліноміальна модель високого ступеня демонструє перенавчання, тоді як поліноміальна модель другого ступеня забезпечує найбільш оптимальний компроміс між точністю та здатністю до узагальнення.

Посилання на [GitHub](#)

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.4	Арк.
						12
Змн.	Арк.	№ докум.	Підпис	Дата		